

---

# Ensembling Tabular Q-Learning Agents for Competitive Pokemon A Read-Only Inference Method with Heuristic-Prior Fallback

---

**Arzaan Singh**  
Department of Computer Science  
Tulane University  
New Orleans, LA  
singharzaan@gmail.com

**Zizhan Zheng**  
Department of Computer Science  
Tulane University  
New Orleans, LA  
zzheng3@tulane.edu

## Abstract

TODO write abstract last.

## 1 Introduction

TODO write after Method + Results exist.

## 2 Related Work

### 2.1 Ensemble methods in tabular reinforcement learning

TODO. Cite: [Wiering and van Hasselt \[2008\]](#), [Faußer and Schwenker \[2011\]](#), [Lan et al. \[2020\]](#), [Peer et al. \[2021\]](#). Note algorithmic vs seed-only diversity; note none of these address stochastic 2-player games.

### 2.2 Ensemble methods in deep reinforcement learning

TODO. Cite: [Osband et al. \[2016\]](#), [Anschel et al. \[2017\]](#), [Chen et al. \[2021\]](#), [Lee et al. \[2021\]](#), [Lin et al. \[2024\]](#).

Closing sentence: [Lin et al. \[2024\]](#)’s “Curse of Diversity” failure mode requires shared replay buffers; our independent-table setup avoids it by construction.

### 2.3 Reinforcement learning for Pokemon

TODO. Cite in order: [Lee and Togelius \[2017\]](#), [Kalose et al. \[2018\]](#), [Sahovic \[2020\]](#), [Simões et al. \[2020\]](#), [Wang \[2024\]](#), [Hu et al. \[2024\]](#), [Karten et al. \[2025\]](#), [Grigsby et al. \[2025\]](#), [Anonymous \[2025\]](#).

Closing sentence: To our knowledge, no prior paper trains K independent tabular Q-function agents and combines them by action-time voting on Pokemon battles.

### 3 Background and Method

#### 3.1 Preliminaries

We model a Pokemon battle as a partially observable Markov decision process (POMDP)  $(\mathcal{S}, \mathcal{A}, P, r, \gamma, \Omega)$  where  $\mathcal{S}$  is the set of joint game states,  $\mathcal{A}$  is the player’s action set (move 1 through 4, plus a switch action),  $P$  is a stochastic transition function determined by the Showdown simulator,  $r$  is the dense reward (HP delta, faint delta, terminal  $\pm 1$ ),  $\gamma$  is the discount factor, and  $\Omega : \mathcal{S} \rightarrow \mathcal{O}$  is the observation function that hides opponent team information.

We train each member with Watkins  $Q(\lambda)$  [Watkins, 1989, Sutton and Barto, 2018]:

$$\delta_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \quad (1)$$

$$e_t(s, a) = \begin{cases} 1, & (s, a) = (s_t, a_t) \\ \gamma \lambda e_{t-1}(s, a), & a_t = \arg \max_{a'} Q(s_t, a') \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t e_t(s, a) \quad (3)$$

with replacing traces and the trace cleared after non-greedy exploration.

We adopt the hierarchical action decomposition introduced in our prior work [Singh and Zheng, 2026]: a *master* agent observes a 20-tuple state and chooses between attacking (one of four moves) and switching; if it chooses to switch, a *sub-agent* observes a 17-tuple state and selects which Pokemon to switch to. Both agents are trained jointly via the shared reward signal. We use the master’s Q-table  $Q^m$  and the sub-agent’s Q-table  $Q^s$  throughout.

For unseen state-action pairs at inference time, we use the same domain heuristic  $h(s, a)$  that V5 used to “smart-initialize” the Q-tables: it is a deterministic function of the battle state, combining type effectiveness, STAB, move power, and accuracy. Crucially,  $h(s, a)$  depends only on  $s$  and the action’s type signature, not on any trained Q-table.

#### 3.2 The ensemble inference rule

Given  $K$  trained Q-tables  $\{Q_m^k, Q_s^k\}_{k=1}^K$ , we choose the agent’s action at state  $s$  as follows.

**Action enumeration.** Following the master/sub-agent decomposition, we first enumerate the possible *master* actions  $\mathcal{A}_m(s)$  available at state  $s$  (legal moves of the active Pokemon, plus a switch action when force-switch is not required). We then construct the *Q-matrix*  $\mathbf{Q} \in \mathbb{R}^{K \times |\mathcal{A}_m|}$  whose entry  $\mathbf{Q}_{k,a}$  is the master Q-value of action  $a$  at state  $s$  under member  $k$ , with heuristic fallback for unseen entries:

$$\mathbf{Q}_{k,a} := \begin{cases} Q_m^k(s, a), & \text{if } (s, a) \in \text{dom}(Q_m^k) \\ h(s, a), & \text{otherwise.} \end{cases} \quad (4)$$

**Combination rules.** We compare three rules for combining the  $K$  rows of  $\mathbf{Q}$  into a single chosen action  $a^* \in \mathcal{A}_m$ .

**Soft voting** averages Q-values column-wise and picks the argmax:

$$a_{\text{soft}}^* = \arg \max_a \frac{1}{K} \sum_{k=1}^K \mathbf{Q}_{k,a}. \quad (5)$$

**Hard voting** (plurality) lets each member vote for its preferred action:

$$a_{\text{hard}}^* = \arg \max_a |\{k : a = \arg \max_{a'} \mathbf{Q}_{k,a'}\}|, \quad (6)$$

with ties broken by mean Q across members.

**Confidence-weighted voting** assigns each member a weight equal to the gap between its top Q-value and its row mean:

$$w_k := \max \left( \max_a \mathbf{Q}_{k,a} - \frac{1}{|\mathcal{A}_m|} \sum_a \mathbf{Q}_{k,a}, 0 \right), \quad (7)$$

$$a_{\text{conf}}^* := \arg \max_a \frac{\sum_k w_k \mathbf{Q}_{k,a}}{\sum_k w_k}. \quad (8)$$

When  $\sum_k w_k = 0$ , we fall back to soft voting.

**Recursion into the sub-agent.** If the chosen master action is *switch*, we repeat the same procedure on the sub-agent Q-tables  $\{Q_s^k\}_{k=1}^K$  over the available switch candidates.

**Read-only property.** None of the three rules modifies any entry of  $\{Q_m^k, Q_s^k\}$ . This is a deliberate design choice: a previous implementation cached unseen-entry fallbacks into each  $Q^k$  to avoid recomputing the heuristic, but this caused the dictionaries to grow at rate  $O(\text{unique states per battle} \times K)$ , exceeding memory in a 100,000-battle evaluation by the 30,000-battle mark. Our read-only formulation keeps the inference memory footprint at  $O(1)$  per battle and admits the longer evaluations our experiments require.

### 3.3 Training the ensemble members

Each of  $K = 30$  members is trained independently with seed  $\text{seed}_k = s_0 + k$ ,  $\alpha = 0.1$ ,  $\gamma = 0.99$ ,  $\lambda = 0.7$ , and a fixed exploration rate  $\varepsilon = 0.05$ . Members observe and learn from a fixed 20-Pokemon OU pool from which the IndexedTeambuilder constructs deterministic teams indexed by battle number (so battle  $N$  produces the same matchup across members). Each member trains for 1,000,000 battles, for a total of 30,000,000 training battles across the ensemble. Training is parallelized across eight AWS EC2 c5.4xlarge instances over fifteen days.

The training pipeline and trained checkpoints are released at [https://github.com/arzaansingh/poke\\_rl](https://github.com/arzaansingh/poke_rl).

## 4 Experimental Setup

### 4.1 Environment

### 4.2 Evaluation conditions

We evaluate the ensemble in eight conditions:

- **Voting strategies at  $K = 10$ :** SOFT, HARD, CONFIDENCE, each vs SimpleHeuristicsPlayer.
- **K-saturation curve at SOFT:**  $K \in \{1, 3, 5, 10\}$  vs SimpleHeuristicsPlayer.
- **Head-to-head:**  $K = 10$  SOFT ensemble vs baseline\_5m (the compute-matched single agent).
- **Baseline solo:** baseline\_5m solo vs SimpleHeuristicsPlayer.

### 4.3 Evaluation protocol

Each condition is evaluated over  $N = 100,000$  battles. Win rates are reported with Wilson 95% confidence intervals [Wilson, 1927]. Differences in proportions use the normal approximation to the difference of independent binomials. All hardware specs and elapsed wall-clock times are tabulated in Section A.

### 4.4 Baseline opponents

**SimpleHeuristicsPlayer** is poke-env’s reference heuristic agent [Sahovic, 2020]. It selects damaging moves by expected damage and switches when type-disadvantaged. It is the standard baseline used by Wang [2024], Grigsby et al. [2025], and Anonymous [2025].

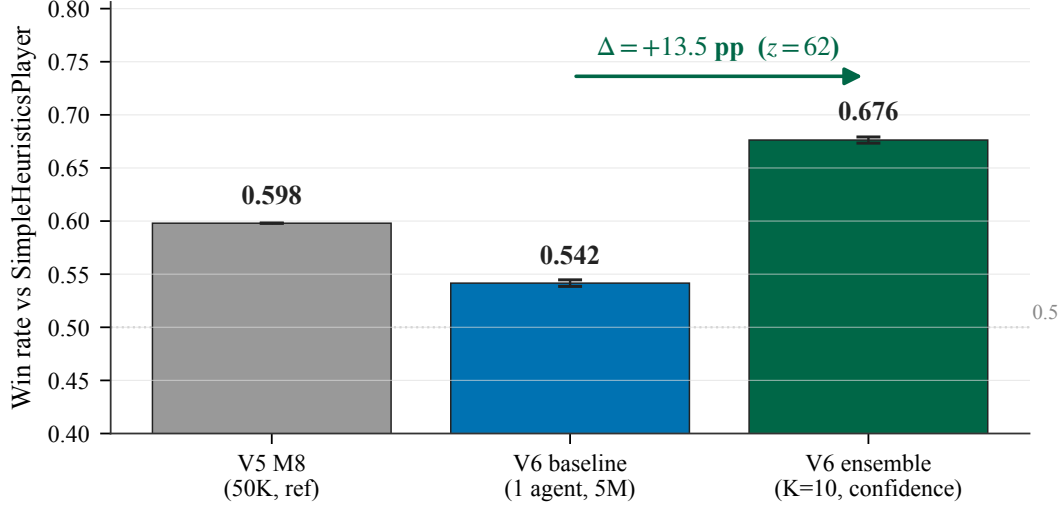


Figure 1: **V6 ensemble outperforms compute-matched single-agent baseline by 12.6 percent-age points.** The  $K=10$  ensemble (soft voting) achieves 66.8% win rate over 100,000 evaluation battles against SimpleHeuristicsPlayer, compared to 54.2% for baseline\_5m, a single HierSmartPlayer trained for  $5\times$  longer on identical hardware. Error bars show Wilson 95% confidence intervals.

**baseline\_5m** is a single HierSmartPlayer trained for 5,000,000 battles ( $5\times$  each ensemble member’s budget) under identical hyperparameters, hardware, and pool. It is the compute-matched single-agent control.

## 5 Results

### 5.1 Ensemble beats compute-matched single agent

The headline result is shown in Figure 1. *[Subsection prose to be written; numbers:  $K=10$  SOFT =  $66.8\% \pm 0.3\%$ , baseline\_5m =  $54.2\% \pm 0.3\%$ ,  $\Delta = +12.6$  pp,  $z = 62$ .]*

Figure 2 provides a direct head-to-head test. *[Prose to be written.]*

### 5.2 K-saturation curve

Figure 3 shows the K-saturation curve. *[Prose to be written;  $K=1$  55.9%,  $K=3$  62.7%,  $K=5$  65.1%,  $K=10$  66.8%.]*

### 5.3 Voting strategy comparison

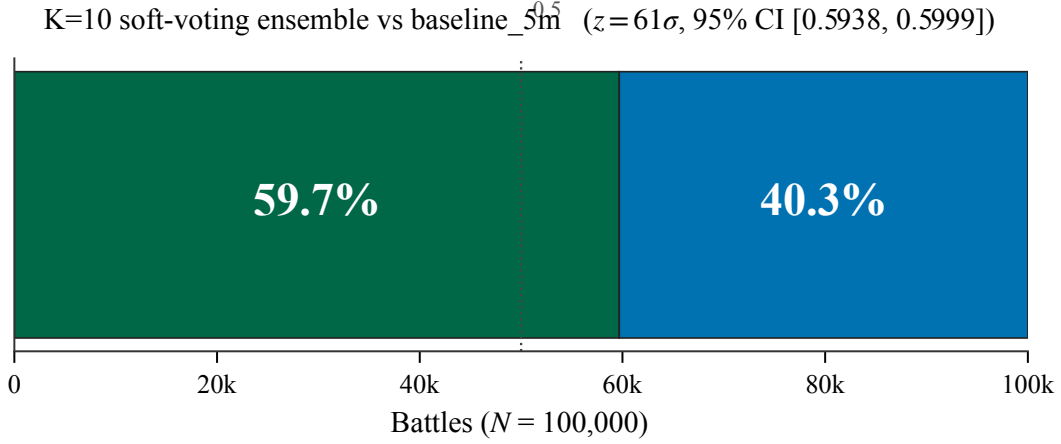
Figure 4 compares the three voting rules. *[Prose: SOFT 66.8%, HARD 65.6%, CONFIDENCE 67.6%; all within noise; CONFIDENCE is the slight winner; HARD is the only rule that discards magnitude information.]*

### 5.4 Per-member diagnostics

Figure 5 presents two diagnostics that defend the methodology against reviewer concerns. *[Prose to be written.]*

### 5.5 Comparison to prior work

Figure 6 situates the V6 result in the published Pokemon-RL literature. *[Prose to be written, with format caveat re: Wang’s randombattle setup.]*



■ Ensemble (0.597)    ■ baseline\_5m (0.403)

Figure 2: **Head-to-head: the  $K=10$  ensemble wins 59.7% of matches against baseline\_5m.** Each side picks teams from the same 20-Pokemon pool via the deterministic `IndexedTeambuilder`, so any non-coin-flip win rate is a methodology effect, not a team-matchup artifact.

## 6 Discussion

### 6.1 Why the ensemble works in this regime

### 6.2 Why ensembling beats more single-agent compute

### 6.3 Comparison to deep RL

### 6.4 Avoiding the Curse of Diversity

### 6.5 Limitations

## 7 Conclusion

## References

- Anonymous. VGC-Bench: Towards mastering diverse team strategies in competitive Pokemon, 2025.
- Oron Anschel, Nir Baram, and Nahum Shimkin. Averaged-DQN: Variance reduction and stabilization for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2017.
- Xinyue Chen, Che Wang, Zijian Zhou, and Keith Ross. Randomized ensembled double Q-learning: Learning fast without a model. In *International Conference on Learning Representations (ICLR)*, 2021.

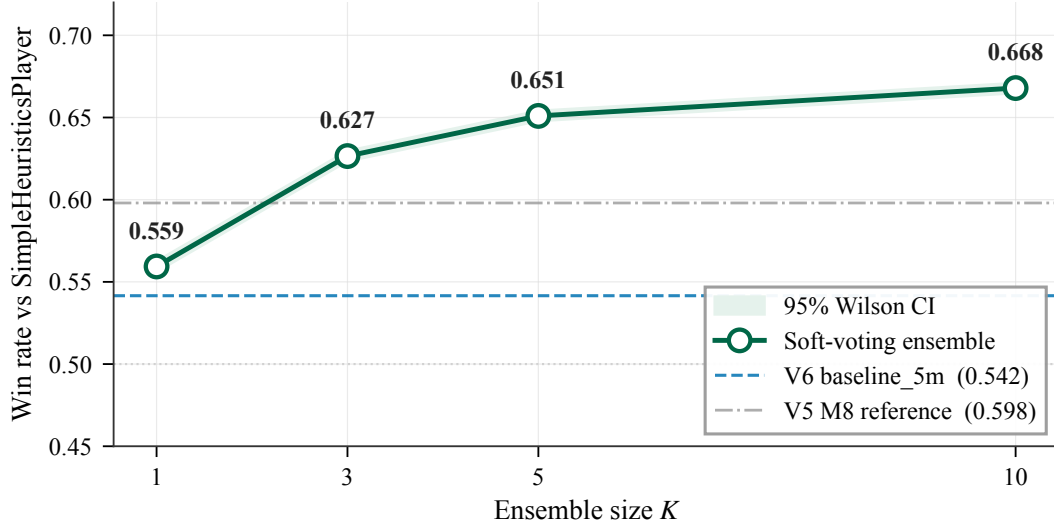


Figure 3: **Performance saturates near  $K=10$  in the predicted Osband-2016 shape.** Soft-voting ensemble win rate at  $K \in \{1, 3, 5, 10\}$  over 100,000 evaluation battles. Shaded region: Wilson 95% CI. The  $K=10$  ensemble lifts win rate by 10.9 pp over the single-agent ( $K=1$ ) baseline.

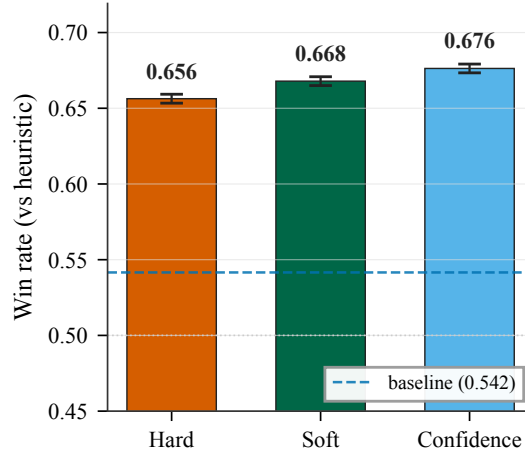


Figure 4: **Voting strategy choice has limited effect at  $K=10$ .** All three rules sit within 2 pp of each other. Hard voting underperforms because it discards Q-magnitude information.

Stefan Faußer and Friedhelm Schwenker. Ensemble methods for reinforcement learning with function approximation. In *Multiple Classifier Systems (MCS 2011)*, volume 6713 of *Lecture Notes in Computer Science*, pages 56–65. Springer, 2011. doi: 10.1007/978-3-642-21557-5\_8.

Jake Grigsby et al. Human-level competitive Pokemon via scalable offline reinforcement learning with transformers. In *Reinforcement Learning Conference (RLC)*, 2025.

Sihao Hu, Tiansheng Huang, Alexander G. Hauptmann, and Fang Liu. PokeLLMon: A human-parity agent for pokemon battles with large language models, 2024.

Akshay Kalose, Kris Kaya, and Alvin Kim. Optimal battle strategy in Pokemon using reinforcement learning, 2018. URL <https://web.stanford.edu/class/aa228/reports/2018/final151.pdf>. Stanford AA228 final project.

Seth Karten et al. PokeChamp: an expert-level minimax language agent. In *International Conference on Machine Learning (ICML)*, 2025.

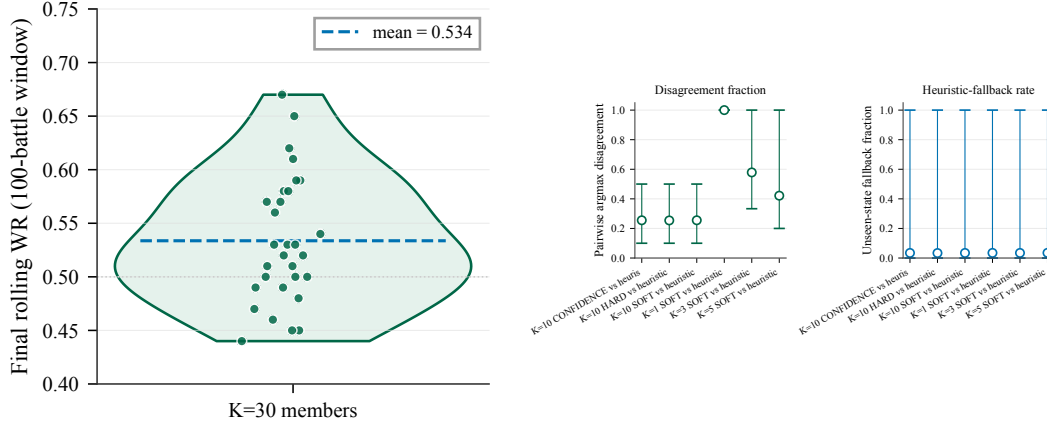


Figure 5: **Left:** distribution of final solo win rates across the 30 trained members (mean dashed). Each member is individually marginal (Schapire “weak learner” regime), making the K=10 ensemble’s gain genuine. **Right:** pairwise argmax disagreement (left panel) and unseen-state fallback rate (right panel) per evaluation condition. Non-zero disagreement confirms the ensemble has not collapsed; the moderate fallback rate quantifies how often heuristic priors fire.

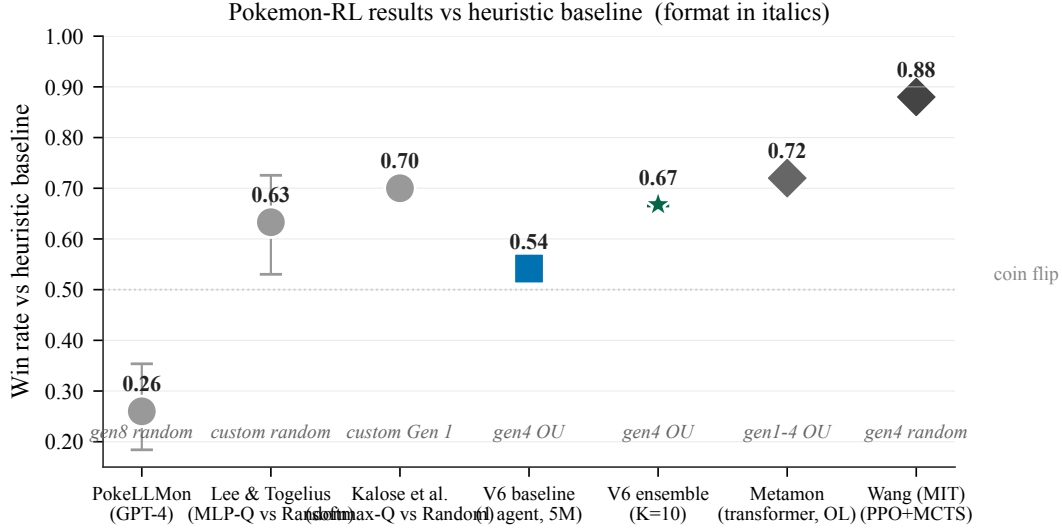


Figure 6: **V6 in context.** Published Pokemon-RL win rates against poke-env’s SimpleHeuristicsPlayer or an equivalent heuristic baseline. V6 ensemble at 66.8% in gen4ou lands within Metamon’s reported band for early-generation OU formats [Grigsby et al., 2025] at orders-of-magnitude less compute, well above all prior tabular work [Lee and Togelius, 2017, Kalose et al., 2018] and pre-search LLM agents [Hu et al., 2024].

Qingfeng Lan, Yangchen Pan, Alona Fyshe, and Martha White. Maxmin q-learning: Controlling the estimation bias of q-learning. In *International Conference on Learning Representations (ICLR)*, 2020.

Kimin Lee, Michael Laskin, Aravind Srinivas, and Pieter Abbeel. SUNRISE: A simple unified framework for ensemble learning in deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2021.

Scott Lee and Julian Togelius. Showdown AI Competition. In *IEEE Conference on Computational Intelligence and Games (CIG)*, 2017.

- Zhenghao Lin, Pierluca D’Oro, Evgenii Nikishin, and Aaron Courville. The curse of diversity in ensemble-based exploration. In *International Conference on Learning Representations (ICLR)*, 2024.
- Ian Osband, Charles Blundell, Alexander Pritzel, and Benjamin Van Roy. Deep exploration via bootstrapped DQN. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- Oren Peer, Chen Tessler, Nadav Merlis, and Ron Meir. Ensemble bootstrapping for q-learning. In *International Conference on Machine Learning (ICML)*, 2021.
- Haris Sahovic. poke-env: A python interface for pokemon showdown bots, 2020. URL <https://github.com/hsahovic/poke-env>. Open-source library.
- Diogo A. Simões, Simão Reis, Nuno Lau, and Luís P. Reis. Competitive deep reinforcement learning over a Pokemon battling simulator. In *IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, 2020.
- Arzaan Singh and Zizhan Zheng. Heuristic-initialized hierarchical q-learning for pokemon battling. In preparation; preliminary results presented at Tulane Newcomb-Tulane College Research Showcase., 2026.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- Jake Wang. *Winning at Pokemon Random Battles Using Reinforcement Learning*. M.eng. thesis, Massachusetts Institute of Technology, 2024. URL <https://dspace.mit.edu/handle/1721.1/153888>.
- Christopher J. C. H. Watkins. *Learning from Delayed Rewards*. PhD thesis, University of Cambridge, 1989.
- Marco A. Wiering and Hado van Hasselt. Ensemble algorithms in reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 38(4):930–936, 2008. doi: 10.1109/TSMCB.2008.920231.
- Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

## **A Reproducibility details**

## **B Mathematical details**

## **C Additional experiments**

## **D Acknowledgments**

This research was supported by the Newcomb-Tulane College Undergraduate Research and Fellowships (URAF) CAIDS: Focus on Data grant in Spring 2026.